

A* (A-STAR) ALGORITHM

AIM

To develop a Python program that implements the A* (A-Star) algorithm to find the shortest path between a start node and a goal node using heuristic search.

OBJECTIVE

- To understand the concept of the A* search algorithm in Artificial Intelligence.
- To find the shortest path from the start node to the goal node.
- To use heuristic values to improve search efficiency.
- To determine the optimal path with minimum cost.

ALGORITHM

Step 1: Start.

Step 2: Initialize the **Open List** with the start node and an empty **Closed List**.

Step 3: Select the node with the lowest $f(n) = g(n) + h(n)$ value from the Open List.

Step 4: If the selected node is the goal node, display the shortest path and stop.

Step 5: Move the selected node from the Open List to the Closed List.

Step 6: Generate all neighboring nodes of the current node.

Step 7: Calculate the cost values:

- $g(n)$ = Cost from the start node.
- $h(n)$ = Heuristic estimate to the goal.
- $f(n) = g(n) + h(n)$.

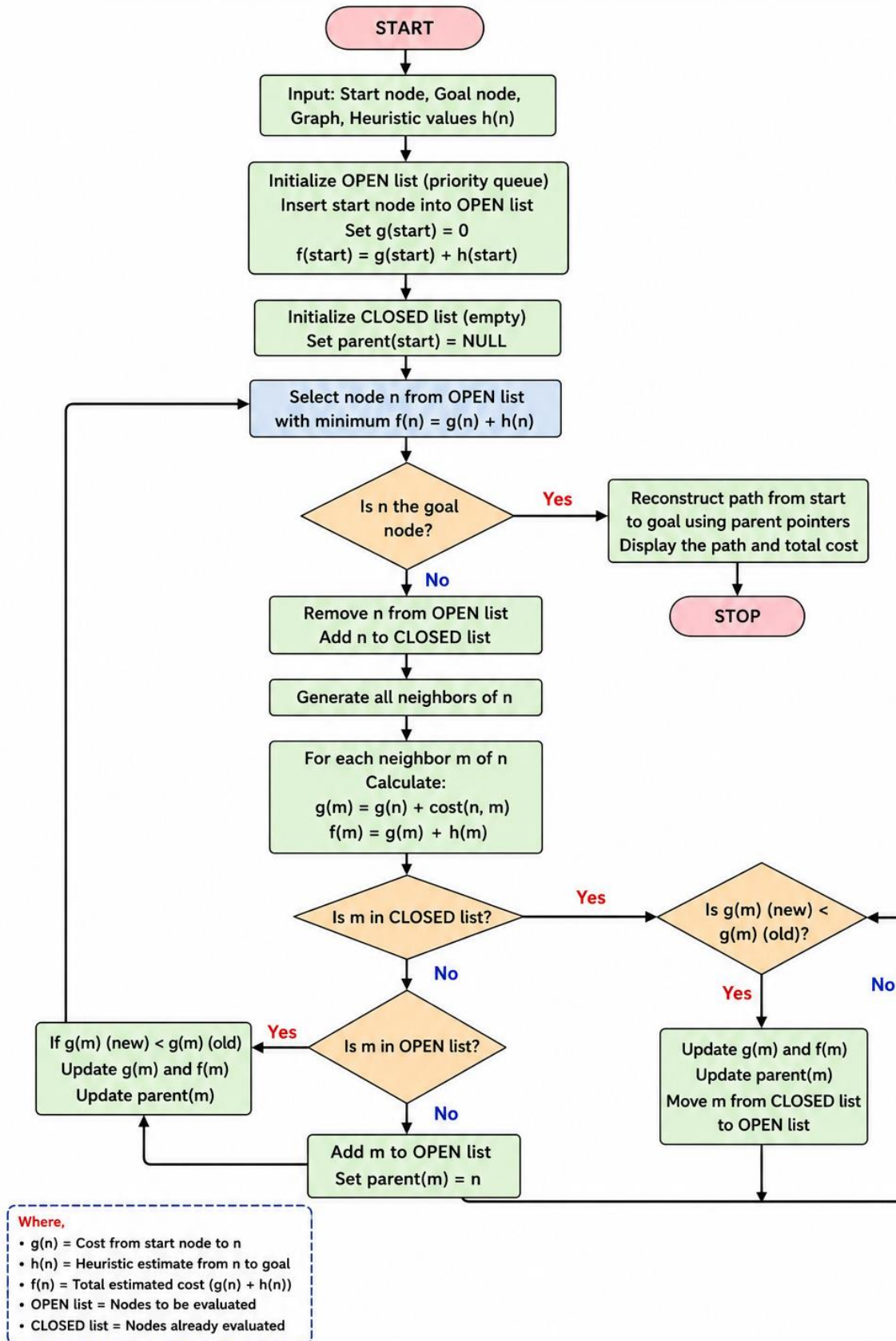
Step 8: Update the Open List with the neighboring nodes having the lowest cost.

Step 9: Repeat Steps 3–8 until the goal node is found or the Open List becomes empty.

Step 10: Stop.

FLOW CHART

A* (A-STAR) ALGORITHM – FLOWCHART



SOURCE CODE

```
import heapq
```

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [('G', 4)],  
    'E': [('G', 2)],  
    'F': [('G', 1)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 4,  
    'E': 2,  
    'F': 1,  
    'G': 0  
}
```

```
def astar(start, goal):  
    open_list = []  
    heapq.heappush(open_list, (0, start))  
  
    g_cost = {start: 0}  
    parent = {start: None}  
  
    while open_list:
```

```

_, current = heapq.heappop(open_list)

if current == goal:
    path = []
    while current:
        path.append(current)
        current = parent[current]
    return path[::-1]

for neighbor, cost in graph[current]:
    new_cost = g_cost[current] + cost

    if neighbor not in g_cost or new_cost < g_cost[neighbor]:
        g_cost[neighbor] = new_cost
        f_cost = new_cost + heuristic[neighbor]
        heapq.heappush(open_list, (f_cost, neighbor))
        parent[neighbor] = current

return None

path = astar('A', 'G')

print("Shortest Path:", path)

```

OUTPUT

```

===== RESTART: C:/Users/Abinaya Sri J/EX9.py =====
Shortest Path: ['A', 'B', 'D', 'G']
|

```

Result

The Python program successfully implements the **A*** (**A-Star**) algorithm and finds the **shortest path** from the start node to the goal node using heuristic values, producing the optimal path efficiently.